

3. HAFTA ARAŞTIRMA

Git ve Bitbucket

Branch:

“Branch” kavramı, bir klasöre benzetilebilir. Bir klasörün olduğunu ve bu klasörü her seferinde kopyalayarak bir “klon” oluşturabildiği düşünülürse her klon üzerinde yapacağınız değişiklikler, ana klasörü etkilemez ve istediğiniz zaman ana klasörden tekrar bir kopya alabilirsiniz. Branch yani dallama stratejileri buna benzer bir biçimde işler. Branch oluşturmak kullanıcıya çalıştığı projenin farklı versiyonlarına erişmesini sağlar. Kullanıcı, projesine bir yenilik eklemek istediğinde, yaptığı değişiklik projenin çalışmasını olumsuz etkileyebilir. Bu gibi durumlarda projemizin o anki halini bozmamak için branch kullanabiliriz. Branch yardımı ile projemizin çalışma halini kaydedip, yeni eklenti üzerinde rahatlıkla çalışabiliriz. Projemizde herhangi bir sorun çıktığı takdirde geri dönüp önceki versiyona kolaylıkla erişebiliriz.

Git’de proje oluşturduğunuzda varsayılan olarak master adında bir branch oluşturulur (ana branch). Eğer hiçbir değişiklik yapmadan çalışmaya devam edersek, bütün işlemlerimizi ana branch’in üzerinden yapmış oluruz. Yeni bir eklenti yapacağımız zaman ana branch’in üzerinde yeni bir branch açabiliriz. Böylece hem dosya kalabalığının hem de dosya boyutunun katlanmasının önüne geçmiş oluruz. Ana branch’ten ayrı olarak, her branch, farklı bir işlevi veya özelliği temsil eder. Bu sayede, farklı ekipler veya geliştiriciler aynı anda farklı özellikleri geliştirebilir ve değişiklikleri çakışmadan izleyebilirler. Ana branch, projenin en güncel ve kararlı halini temsil eder. Yeni bir özellik eklemek veya bir hata düzeltilmesi yapmak istendiğinde, geliştiriciler ana branch’i değiştirmeden kendi branch’lerinde çalışır. Daha sonra bu değişiklikler ana branch’e entegre edilir, bu işleme de “birleştirme” (merge) denir. Bu sayede, geliştiricilerin birbirlerinin çalışmalarını etkilemeden verimli bir şekilde geliştirme yapmaları mümkün olur.

Farklı merge çeşitleri farklı özelliklerdedir.

Fast Forward Merge: Tek bir dal üzerinden ilerler. Fast Forward Merge kullanırken bir şart vardır o da Master Branch’te yapılacak değişiklikten önce herhangi bir değişiklik yapılmamasıdır. Böylece sapma olmadan yeni değişiklik eklenir ve master branch artık yeni dal dahil hal olur.

No Fast Forward Merge: Yeni branchta yapılan değişiklikler farklı bir düzlem üzerinde ilerler. Her seferinde Git yeni bir birleşme noktası oluşturur. Yapılan her değişiklik ve aşamaları ayrıca tarihlenir ve saklanır.

Three Way Merge (Automatic Merge): Hedef dalda da (master) yeni commit’ler atılmışsa, tarihçe iki ayrı kola ayrılmış demektir. Bu durumda işaretçiyi sadece ileri sarmak imkansızdır. Git, bu iki kolu birleştirmek için arka planda üç noktayı (three-way) referans alır: İki dalın ayrıldığı ilk ortak atası (common ancestor), Kaynak dalın son bulunduğu commit, Hedef dalın son bulunduğu commit. Bunları otomatik birleştirir.

Rebase:

Rebase, Git'te bir daldaki commit'leri başka bir dala yeniden yazmanıza olanak tanıyan bir işlemdir. Git rebase, değişiklikleri entegre etmek için commit'lerinizi başka bir dalın en son durumunun üzerine yeniden oynatır ve daha temiz, doğrusal bir geçmiş oluşturur. Değişiklikleri hedef dalın en üst noktasına taşır. Birleştirme taahhüdü oluşturmak yerine taahhüt geçmişini yeniden yazar. Proje geçmişini doğrusal ve okunması daha kolay hale getirir.

Pull Request:

Bir pull request yani çekme isteği (veya birleştirme isteği) sırasında , depo yöneticisi, geliştiriciden gelen yeni kod güncellemelerini inceleyerek yayınlanmaya hazır olup olmadığını belirler. Çekme istekleri olmadan, tamamlanmamış veya yanlış yazılmış kod güncellemeleri ana depoya erken bir şekilde birleştirilebilir ve canlı üründe sorunlara veya arızalara neden olabilir. Çekme istekleri, yalnızca uygun şekilde incelenmiş ve onaylanmış kodu birleştirerek ürünün bütünlüğünü ve kullanıcı deneyimini sağlar. Projenin deposu (veya "repo"), tüm proje kodunun, ürün bilgisinin, meta verilerinin ve daha birçok veri/bilginin saklandığı yerdir. Depo yöneticisi/bakımcısı, DevOps veya operasyon ekibinde yer alan ve söz konusu depoyu sürdürmekten sorumlu kişidir. Hangi güncellemelerin nihai projeye entegre edilebileceğinden ve nihayetinde son kullanıcılara sunulabileceğinden sorumludurlar. Ana proje deposunu korumanın yanı sıra, çekme istekleri (pull request'ler) yeni ürün güncellemeleri üzerinde çalışırken iş birliğini ve açık iletişimi teşvik eder.

Code Review:

Yazılım geliştirme sürecinde kod inceleme (code review), kaliteyi artırmak ve hataları erken aşamada tespit etmek için vazgeçilmez bir adımdır. Kod inceleme, bir yazılım projesinde geliştirilen kodun, ekip üyeleri tarafından gözden geçirilmesi ve değerlendirilmesi sürecidir “two heads are better than one” şeklindeki basit varsayıma dayanmaktadır. Code review yalnızca hata bulmak için yapılmaz. Aynı zamanda kodun okunabilirliğini, performansını, güvenliğini, genel tasarımını, test edilebilirliğini ve sürdürülebilirliğini değerlendiren önemli bir yazılım geliştirme tekniğidir.

Helm

Kubernetes üzerinde bir uygulama ayağa kaldırmak, birbirine bağımlı çok sayıda YAML yapılandırma dosyasının (Deployment, Service, Ingress, ConfigMap vb.) oluşturulmasını gerektirir. Helm'in temel amacı; giderek karmaşılaşan bu YAML dosyası yığını tek bir merkezden organize etmek, versiyonlamak, yönetmek ve dağıtım sürecini standartlaştırmaktır.

Helm Chart:

Helm Chart, Helm'in bu karmaşık yapılandırmaları yönetmek için kullandığı paketleme formatıdır. Bir uygulamanın Kubernetes üzerinde çalışabilmesi için ihtiyaç duyduğu tüm kaynak atamalarını, kural tanımlarını ve yapılandırma dosyalarını tek bir mantıksal birim (paket) altında toplar. Chart'lar sayesinde mimari yapılar taşınabilir, paylaşılabilir ve farklı projelerde tekrar tekrar kullanılabilir hale gelir.

Template:

Kubernetes manifestoları normal şartlarda statik YAML dosyalarıdır; yani içindeki veriler sabittir. Template (Şablon) mimarisi, bu statik dosyaları dinamik ve programlanabilir hale getiren temel bileşendir. Template'ler, içlerinde sabit veriler yerine yer tutucular (değişkenler) barındıran iskelet dosyalardır. Helm sisteminin en güçlü yanı, şablon (template) ile veriyi (values) birbirinden ayırmasıdır:

Template Dosyaları: Uygulamanın Kubernetes üzerindeki mimari yapısını ve kurallarını belirler, ancak çevresel detayları içermez.

Values (Değerler) Dosyası: Port numaraları, IP adresleri, kullanıcı adları, şifreler, Docker imaj etiketleri ve kaynak limitleri gibi ortama göre değişebilen parametreleri barındıran harici yapılandırma dosyasıdır.

Helper:

Helm, templates/ dizinindeki dosyaları Kubernetes manifest dosyasına işlenmesi gereken şablonlar olarak kabul eder . Bazı istisnalar vardır

Bu dosyalar, Helm dokümantasyonunda "kısmi dosyalar" ve "yardımcı dosyalar" olarak adlandırılan şeyleri depolamak için kullanılır. Burada depoladığımız şey, bir bakıma programlamadaki Helm şablon fonksiyonlarına benzer . Bir nevi kütüphane oluşturabilir, ihtiyaç duyduğumuz fonksiyonları burada bir kez tanımlayabilir ve ardından aynı fonksiyonları tekrar tekrar yazmadan, bunları gerektiren her şablon dosyasında kullanabiliriz. Yardımcı dosyalar, yeniden kullanılabilir kod içeren ve özellikle bir grafikte tekrar tekrar kullanılan karmaşık Kubernetes nesnelerini tanımlamak için yararlı olan dosyalardır. Geliştiriciler, bu nesneleri bir yardımcı dosyada tanımlayarak kod tekrarından kaçınabilir, daha özlü ve okunabilir şablonlar oluşturabilir ve daha temiz bir Helm grafik yapısı koruyabilirler.

Release:

Helm de Release bir Helm Chart'ın bir Kubernetes kümesine (cluster) ve belirli bir isim alanına (namespace) kurulmuş (deploy edilmiş) çalışır vaziyetteki özel bir kopyasıdır. Chart şablon iken, Release o şablonun hayata geçirilmiş ve kimlik kazanmış halidir.

Upgrade ve Rollback:

Upgrade (yükseltme) ve rollback (geri alma), çalışan uygulamaların sürümlerini yönetmek için kullanılan iki temel yaşam döngüsü işlemidir. Upgrade mevcut bir sürümü yeni ayarlarla günceller Yeni bir konfigürasyon uygulandığında Helm, Kubernetes üzerindeki kaynakları günceller ve bunu geçmişe yeni bir revision (sürüm numarası) olarak kaydeder. Rollback ise sorun anında eski çalışan sürüme güvenle

dönmeyi sağlar. Aslında geçmişe doğru bir silme işlemi yapmaz. Eski kararlı konfigürasyonu alır ve bunu ileriye doğru yeni bir revision olarak tekrar uygular.

ArgoCD

GitOps:

GitOps, altyapı ve uygulama konfigürasyonlarını Git altyapısı ile yönetmemize kolaylık sağlayan bir yaklaşımdır. GitOps ile, altyapı ve uygulamalarımızı tek bir kaynaktan sürdürülebilir bir şekilde yönetebiliriz. GitOps araçları ile birlikte, Git reposunda yer alan konfigürasyon dosyaları ile Kubernetes cluster'ı üzerinde çalışan objeler arasında bir farklılık görüldüğünde uyarı verilmesi veya otomatik senkronizasyon sağlanabilir. GitOps bazı prensiplerden oluşur:

Declarative altyapı: Tüm kaynaklar tanım dosyalarıyla oluşturulur, değişiklikler dosyalar üzerinden yapılır (örn. Kubernetes manifestoları). Tek Git reposunda tutulması Single Source of Truth sağlar; kolay deploy, rollback ve felaket kurtarma imkanı verir.

Git deposu: Tüm tanım dosyaları tek bir Git reposunda yönetilir. git revert ile kolay rollback sağlanır.

Otomatik uygulama + onay: Git değişiklikleri, belirlenmiş onay aşamalarından geçtikten sonra otomatik olarak cluster'a yansır. Yetki gerektirmez, güncellemeler merkezi ve kontrollüdür.

GitOps agent'lar: Agent'lar, reposundaki istenen durum ile cluster'daki gerçek durumu karşılaştırır. Fark varsa alarm oluşturur veya isteğe bağlı otomatik senkronizasyon yapar.

ArgoCD:

Argo CD, Kubernetes ortamları için kullanılan bir GitOps Continuous Deployment aracıdır. Argo CD, GitOps'un temel yapıtaşı olan Single Source of Truth modelini referans alır. Argo CD belirli periyotlarda Kubernetes'e deploy edilmiş uygulamaları izler ve Git reposunda yer alan tanım dosyalarıyla karşılaştırmalar yapar. Herhangi bir farklılık durumunu kullanıcıyı bildirir veya istenilirse otomatik olarak senkronizasyon işlemini gerçekleştirir.

Auto Sync

Argo CD, Git'teki istenen manifest dosyaları ile kümedeki canlı durum arasında farklılıklar tespit ettiğinde uygulamayı otomatik olarak senkronize etme özelliğine sahiptir. Otomatik senkronizasyonun bir avantajı, CI/CD işlem hatlarının artık dağıtımı gerçekleştirmek için Argo CD API sunucusuna doğrudan erişime ihtiyaç duymamasıdır.

Self Healing

Birisi veya bir şey kümedeki bir kaynağı doğrudan değiştirdiğinde (kubectl, Kubernetes kontrol paneli veya başka bir araç kullanarak), ArgoCD sapmayı algılar ve Git ile

eşleştirecek şekilde değişikliği geri alır. Argo CD'de Sync Policy (Senkronizasyon Politikası) ise Git deposundaki tanımlar ile Kubernetes kümesindeki (cluster) canlı durumun nasıl ve ne zaman eşitleneceğini belirleyen kurallar bütünüdür.

Jenkins

Pipeline:

Pipeline işleyişi, bir bilgisayar işlemcisinin bilgisayar komutlarını ayrı aşamalar halinde yürütme sürecidir. Buradaki pipeline hattı, işlemcinin bir komutu birden fazla adımda gerçekleştirmesine olanak tanıyan mantıksal bir yapıdır. Farklı aşamalarındaki çeşitli komutlar, aynı anda boru hattı içinde bulunabilir. İşlem süreci; kesintisiz, düzenli ve kısmen örtüşen bir biçimde gerçekleşir. Bilişim alanında bu yöntem, pipeline processing olarak da adlandırılır. Bazı parçaların diğerlerinden önce monte edilmesi gerekebilse de, farklı ürün bileşenlerinin eş zamanlı olarak birleştirildiği bir üretim montaj hattına benzetilir. Sıralı bir bağımlılık söz konusu olsa bile, birçok işlem eş zamanlı olarak yürütülebilir ve bu da zaman tasarrufu sağlar. Jenkins'te stage (aşama), bir pipeline (işlem hattı) sürecini mantıksal bölümlere ayıran ve her biri belirli bir görevi (derleme, test etme, dağıtma gibi) temsil eden ana adım bloklarıdır.

Jenkinsfile:

Jenkinsfile, Jenkins Pipeline'ın temelini oluşturan ve iş sürecini kodla tanımlayan bir dosyadır. Projenizin kök (root) dizininde yer alır ve job'ların otomatik olarak yürütülmesini sağlar. Jenkinsfile genellikle Groovy programlama dilini kullanarak yazılır ve Jenkins sunucusu tarafından yorumlanır. Jenkinsfile, deklaratif (declarative) veya betik (scripted) söz dizimlerini kullanabilir. Declarative sözdizimi, yapılandırma yönetimini basitleştirir ve daha okunabilir bir formatta sunar. Scripted sözdizimi ise daha esnek bir yapı sağlar ve daha fazla programlama yeteneği sunar.

Harbor ve Nexus

OPI Registry:

Docker'dan doğan Açık Konteyner Girişimi (OCI) konteyner formatı, bulut odaklı konteynerleştirilmiş yazılım dağıtımları için baskın standarttır. Apptainer'ın kendi konteyner formatının birçok benzersiz avantajı olmasına rağmen, bir noktada Docker/OCI konteynerleriyle çalışmanız muhtemeldir. Apptainer, paylaşımlı sistemlerde ve özellikle yüksek performanslı bilgi işlem (HPC) ortamlarında kullanıma uygun bir çalışma ortamının gereklilikleri çerçevesinde, Docker ile maksimum uyumluluğu hedeflemektedir. Docker image repository (imaj deposu), Docker imajlarının saklandığı, sürümlendirildiği ve paylaşıldığı merkezi depolama alanıdır.

Artifact Repository:

DevOps ekipleri için, yapıt deposu (artifact repository) , yazılım geliştirme sürecinde üretilen çeşitli çıktıları ve bağımlılıkları yönetmede çok önemli bir rol oynar. Bu depolar, yapıtlar için merkezi depolama alanları görevi görerek, bunların kolayca erişilebilir, sürüm kontrollü ve geliştirme ve dağıtım süreçlerinin farklı aşamalarında tutarlı olmasını sağlar.

Yazılım geliştirme yaşam döngüsü (SDLC) boyunca üretilen yapıtları depolamak ve yönetmek, yapıt deposunun temel amacıdır. Bu yapıtlar derlenmiş kod, kütüphaneler, bağımlılıklar, yapılandırma dosyaları, dokümantasyon ve derleme çıktılarını içerebilir. Bu bileşenleri merkezileştirerek, yapıt depoları kodlama, test etme, derleme, yayınlama ve dağıtım süreçleri de dahil olmak üzere yazılım geliştirme yaşam döngüsünün her aşamasıyla sorunsuz entegrasyonu kolaylaştırır.

Vulnerability Scan:

Güvenlik açığı, bir BT varlığının veya ağının yapısında, işlevinde veya uygulamasında bulunan herhangi bir zayıflıktır. Bilgisayar korsanları veya diğer tehdit aktörleri, yetkisiz erişim elde etmek ve ağa, kullanıcılara veya işletmeye zarar vermek için bu zayıflıktan yararlanabilir. Yaygın güvenlik açıkları şunlardır:

Kodlama hataları, örneğin, kullanıcı girdilerini ele alma biçimleri nedeniyle siteler arası komut dosyası çalıştırma (XSS), SQL enjeksiyonu ve diğer enjeksiyon saldırılarına karşı savunmasız olan web uygulamaları.

Sunucularda, dizüstü bilgisayarlarda ve diğer uç noktalarda bulunan, bilgisayar korsanlarının kötü amaçlı yazılım yaymak için kullanabileceği korumasız açık bağlantı noktaları .

Yanlış yapılandırmalar, örneğin hassas verileri herkese açık internete maruz bırakan uygunsuz erişim izinlerine sahip bir bulut depolama kovası gibi durumlar .

Eksik yamalar, zayıf parolalar veya siber güvenlik hijyenindeki diğer eksiklikler.

>>Maven / npm / Docker repository farklarının açıklanması:

Maven ve npm paket yöneticileridir. Bunlara projemiz için gerekli kodlar özellikler gibi gereklilikler için ihtiyaç duyarız. Kendi yazılımımızı geliştirirken başkalarının yazdığı hazır kod parçacıklarını, kütüphaneleri projene dahil etmeni, bu kütüphanelerin versiyonlarını yönetmeni ve uygulamanın inşa sürecini düzenlemeni sağlarlar. Docker depoları (repository) ise bir sistemin yapılandırılmış halini tüm gereklilik ve parçalarıyla birlikte barındıran konteyner yapılarıdır. Docker deposu, sadece kodu değil; işletim sistemi ayarlarını, runtime araçlarını ve hatta Maven/npm ile indirilmiş paketlerin tamamını tek bir çalışabilir imaj olarak saklar.

Harbor ve Nexus:

Hem harbor hem de Sonatype Nexus Repository birer "depo" (repository/registry) yöneticisidir, ancak odak noktaları ve uzmanlık alanları birbirinden farklıdır.

Nexus sadece docker imajlarını değil Maven (Java), npm (JavaScript), PyPI (Python) ve NuGet (.NET) gibi aklınıza gelebilecek hemen her türlü paketi tek bir merkezden yönetmenizi sağlar. Tüm yazılım malzemelerinizi tek bir çatı altında toplar.

Harbor konteynerler için özel olarak tasarlanmış yüksek güvenli bir limandır. Nexus gibi her dili desteklemek yerine, sadece Docker imajlarına ve Kubernetes ortamları için Helm chart'larına odaklanır. Bu dar odağı sayesinde, konteyner zafiyet taraması (vulnerability scanning) ve detaylı yetkilendirme konularında çok derinlemesine

özellikler sunar. Çalıştığınız işte aynı anda pek çok dil kullanılıyorsa tüm kütüphaneleri tek mekezde toplamak ve yönetmeyi kolaylaştırmak için nexus kullanabilirsiniz. Yine nexus sayesinde dış dünyadaki paketleri çekip önbellege alarak indirme süresini kısaltmak mümkündür. Ama uygulamalarınız konteynerler ve Helm chartlar üzerinden çalışıyorsa harbor tam ve ideal uyum sağlar. Canlıya çıkacak imajın içinde kritik bir güvenlik açığı varsa yayını otomatik engellemek veya imzasız imajların çalışmasını önlemek istediğinizde Harbor öne çıkar.

AWS, Terraform ve Monitoring

Amazon EC2 (Elastic Compute Cloud) - Bulutta güvenli, yeniden boyutlandırılabilir işlem gücü sağlar. EC2, uygulamalarınızı dağıtmanız için fiziksel bir sunucuyu temsil eden sanal makinedir. Amazon, kendi donanımınızı satın alıp onu bir ağa bağlamanız yerine, uygulamalarınızı çalıştırmanız için size tek bir fiziksel donanımda birden çok sanal bilgisayar sunar. AWS EC2 bulut sunucusu tipleri:

General purpose instances: İşlem, bellek ve ağ kaynakları arasında bir denge sağlar ve çeşitli iş yükleri için kullanılabilir.

Compute optimized instances: Yüksek performanslı işlemcilere ihtiyaç duyan hesaplamaya bağlı uygulamalar için kullanışlıdır.

Storage optimized instances: Belleğe yazılması gereken ve yüksek, sıralı okuma ve yazma erişimi gerektiren çok büyük veri kümeleri içeren iş yükleri için tasarlanmıştır.

Memory optimized instances: Bellekteki büyük veri kümelerini işlemek için hızlı performans sağlar.

Accelerated computing instances: CPU hızı arttığında hızlandırılmış bir performans sağlarlar.

Amazon S3 (Simple Storage Service) - Veri yedekleme, arşivleme ve analiz için ölçeklenebilir nesne depolama sağlar. S3, kullanım kolaylığı, ölçeklenebilirlik, yüksek dayanıklılık (%99,999999999 dayanıklılık) ve güvenlik sunar.

Amazon VPC (Sanal Özel Bulut) - AWS bulutunda özel, yalıtılmış bir bölüm oluşturmanıza olanak tanır. AWS kaynaklarını tanımladığınız bir sanal ağda başlatmanıza olanak tanır. Bu sanal ağ, AWS'nin ölçeklenebilir altyapısını kullanmanın avantajlarıyla kendi veri merkezinde çalıştıracağınız geleneksel bir ağa çok benzer. Farklı VPC'ler VPC peering veya Transit Gateway ile birbirlerine bağlanabilirler. VPC subnetlerden oluşur. VPC'nin içinde subnet oluşturulduğunda, her availability zone (AZ) için bir public, iki de private subnet oluşturabilir.

Terraform:

HashiCorp Terraform, bulut ve şirket içi kaynakları, sürümlendirebileceği, yeniden kullanılabilir ve paylaşılabilen, insan tarafından okunabilir yapılandırma dosyaları tanımlamaya olanak tanıyan bir altyapı kodlama aracıdır. Altyapının tüm yaşam döngüsü boyunca tutarlı bir iş akışı kullanarak sağlanabilir ve yönetilebilir.

Terraform ve Kubernetes, hem uygulama dağıtımı hem de yaşam döngüsü yönetiminde kullanılan DevOps araçlarıdır. Ancak ikisinin de farklı işlevleri vardır.

Terraform, geliřtiricilerin bulut BT kaynaklarını otomatik olarak oluřturmak, saęlamak ve yönetmek için kullandıkları Kod Olarak Altyapı (IaC) aracıdır. Bununla, bulut altyapınızdan istediklerinize odaklanabilir ve kurulum için gerekli adımları otomatik olarak gerçekleřtirebilirsiniz. Buna karřılık Kubernetes, container'larınızı uygun ölçeğe yönetmenize yardımcı olan bir container düzenleme aracıdır. Kaynak tedariki, container zamanlaması, gruptama ve dięer koordinasyon görevlerini yönetir.

KAYNAKLAR

1. <https://novatorsoft.com/blog/branch-nedir-nasil-kullanilir>
2. <https://dev.to/koraybarkin/git-de-branch-nedir-nasil-olusturulur-em6>
3. <https://medium.com/@ofkrc/brenching-ve-merging-dallanma-ve-birle%C5%9Firme-e197a3c79133>
4. <https://www.gitkraken.com/learn/git/git-rebase>
5. <https://www.geeksforgeeks.org/git/rebasing-of-branches-in-git/>
6. <https://www.pagerduty.com/resources/continuous-integration-delivery/learn/what-is-a-pull-request/>
7. <https://tigonproject.com/tr/kod-inceleme-code-review-nedir>
8. <https://archisacademy.com/tr/blogs/kod-incelemesi>
9. <https://www.freecodecamp.org/news/what-is-a-helm-chart-tutorial-for-kubernetes-beginners/>
10. <https://medium.com/@mesutpiskin/helm-nedir-helm-ile-kubernetesa-deployment-nas%C4%B1l-yap%C4%B1%C4%B1r-8835d0f5d976>
11. <https://circleci.com/blog/what-is-helm/#c-consent-modal>
12. <https://kodekloud.com/blog/helper-files-and-named-templates/>
13. https://helm.sh/docs/helm/helm_rollback/
14. <https://medium.com/devopsturkiye/gitops-argocd-2a4e771311af>
15. <https://oneuptime.com/blog/post/2026-02-26-argocd-self-healing-vs-manual-sync/view>
16. https://argo-cd.readthedocs.io/en/release-2.6/user-guide/auto_sync/
17. <https://www.techtarget.com/whatis/definition/pipelining>
18. <https://medium.com/@mert-kilicaslan/jenkinsfile-nedir-daffe79022ac>
19. https://apptainer.org/docs/user/main/docker_and_oci.html
20. <https://jfrog.com/learn/devops/software-artifact-repository/>
21. <https://www.ibm.com/think/topics/vulnerability-scanning>
22. https://gemini.google.com/app?is_sa=1&is_sa=1&android-min-version=301356232&ios-min-version=322.0&campaign_id=bkws&utm_source=sem&utm_source=google&utm_medium=paid-media&utm_medium=cpc&utm_campaign=bkws&utm_campaign=2024trTR_gemfeb&pt=9008&mt=8&ct=p-growth-sem-bkws&gclidsrc=aw.ds&gad_source=1&gad_campaignid=22165901653&gbraid=0AAAAApk5Bhmv0_AX5m3hTOUJUvUAmpK65&gclid=Cj0KCQjw-4XFBhCBARIsAAAdNOKsO92EBA4atVhC4yB3S6Qm9FNxfcB1PhlPIfkMvsTfljw61luD9hl4aAj50EALw_wcB
23. <https://awsforengineers.com/blog/aws-tutorial-for-beginners-step-by-step-core-concepts/>
24. <https://miuul.com/blog/awse-dair-temel-kavramlar>
25. <https://aws.amazon.com/tr/compare/the-difference-between-terraform-and-kubernetes/>
- 26.